

Explicación del código

Detección de objetos en tiempo real con YOLOv8

Guía técnica y conceptual del notebook

`NombreApellido_DeteccionObjetos.ipynb`

Tecnologías principales

Python · Ultralytics YOLOv8 Nano · OpenCV · PyTorch · Pandas · Jupyter

28 de julio de 2026

Índice

1.	Propósito del programa	1
1.1.	Archivos del proyecto	1
2.	Tecnologías y responsabilidades	1
3.	Flujo general	2
4.	Instalación de las bibliotecas	2
4.1.	Qué hace cada instrucción	2
5.	Configuración, importaciones y carga del modelo	2
5.1.	Configuración local de Ultralytics	2
5.2.	Carga de pesos	3
5.3.	Selección del dispositivo	3
6.	Función <code>abrir_camara</code>	3
7.	Función <code>construir_tabla</code>	3
7.1.	Sistema de coordenadas	4
8.	Función principal: <code>detectar_en_tiempo_real</code>	4
8.1.	Apertura y validación	4
8.2.	Lectura continua	5
8.3.	Inferencia con YOLO	5
8.4.	Dibujo e interfaz	5
8.5.	Lectura del teclado	6
8.6.	Liberación segura de recursos	6
8.7.	Presentación en Jupyter	7
9.	Las tres pruebas	7
10.	Verificación final	8
11.	Parámetros que modifican el comportamiento	8
12.	Manejo de errores y robustez	9
13.	Limitaciones y mejoras recomendadas	9
13.1.	Limitaciones actuales	9
13.2.	Mejoras posibles	9
14.	Cómo ejecutar correctamente el notebook	10
15.	Resumen conceptual	10
	Glosario breve	10

1. Propósito del programa

El notebook implementa un sistema local de **visión artificial** que toma video de la cámara de la computadora, analiza cada fotograma con el modelo **YOLOv8 Nano** y muestra los objetos encontrados. Para cada detección presenta tres datos:

- la clase u objeto reconocido, por ejemplo `person`, `bottle` o `cell phone`;
- la confianza estimada por el modelo;
- un recuadro delimitador que indica la posición del objeto en la imagen.

El programa también permite capturar una evidencia con la tecla **C**. La captura se conserva en memoria, se muestra dentro del notebook y se acompaña de una tabla con las detecciones. Las teclas **Q** y **Esc** cierran la prueba sin guardar evidencia.

1.1. Archivos del proyecto

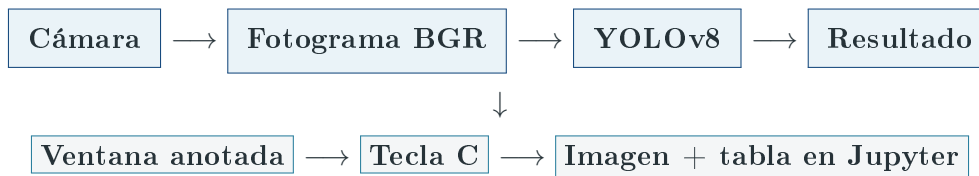
Archivo o carpeta	Función
NombreApellido_ DeteccionObjetos.ipynb	Notebook que contiene las explicaciones, instalación, funciones, pruebas y verificación final.
yolov8n.pt	Pesos preentrenados de YOLOv8 Nano. Contienen los parámetros que el modelo aprendió antes de usarse en este proyecto.
.yolo_config	Carpeta local destinada a la configuración de Ultralytics, creada para evitar problemas de permisos en carpetas globales.

2. Tecnologías y responsabilidades

Componente	Responsabilidad en el código
Python	Coordina el flujo completo y conecta las bibliotecas.
Jupyter/IPython	Ejecuta el programa por celdas y muestra imágenes, tablas y texto enriquecido en la salida.
Ultralytics YOLO	Carga el modelo y ejecuta la inferencia sobre cada fotograma.
PyTorch	Proporciona el motor numérico usado por YOLO y permite comprobar si hay una GPU CUDA disponible.
OpenCV (cv2)	Abre la cámara, obtiene fotogramas, dibuja texto, muestra la ventana y lee las teclas.
NumPy	Representa las imágenes como arreglos numéricos; se importa como <code>np</code> , aunque el notebook no lo usa directamente.
Pandas	Convierte las detecciones en una tabla legible.
Pillow (PIL)	Convierte el arreglo RGB en una imagen que Jupyter puede mostrar.

Nota. YOLO significa *You Only Look Once*. Es un detector de una sola etapa: procesa la imagen y produce clases, confianzas y posiciones en una misma inferencia. La variante Nano se eligió por ser ligera y adecuada para una demostración en tiempo real.

3. Flujo general



El ciclo se repite mientras la ventana está abierta. En cada vuelta se lee una imagen, se ejecuta el modelo, se dibujan los resultados y se consulta el teclado. Al capturar o salir, el bucle termina y los recursos de la cámara se liberan.

4. Instalación de las bibliotecas

La primera celda de código utiliza el comando especial `%pip`. En Jupyter, este comando instala paquetes en el entorno asociado con el kernel actual.

```

1 %pip install -q --upgrade --no-deps \
2   "torch==2.11.0+cpu" "torchvision==0.26.0+cpu" \
3   --index-url https://download.pytorch.org/whl/cpu
4
5 %pip uninstall -y opencv-python-headless
6 %pip install -q --upgrade --force-reinstall --no-deps \
7   "opencv-python>=4.8,<5"
8 %pip install -q "ultralytics>=8.3,<9" pandas pillow

```

4.1. Qué hace cada instrucción

- Instala versiones de `torch` y `torchvision` preparadas para CPU. La opción `-no-deps` evita que `pip` modifique dependencias adicionales durante ese paso.
- Desinstala `opencv-python-headless`. Esa variante está pensada para servidores sin interfaz gráfica y no ofrece la ventana requerida por `cv2.imshow`.
- Reinstala la variante normal de OpenCV, que sí puede abrir ventanas.
- Instala Ultralytics, Pandas y Pillow. Los intervalos de versiones limitan cambios incompatibles.

Atención. Después de modificar PyTorch u OpenCV conviene reiniciar el kernel. Un proceso de Python ya iniciado puede conservar en memoria la versión anterior de una biblioteca.

5. Configuración, importaciones y carga del modelo

5.1. Configuración local de Ultralytics

```

1 carpeta_configuracion = Path.cwd() / ".yolo_config"
2 carpeta_configuracion.mkdir(exist_ok=True)
3 os.environ.setdefault("YOLO_CONFIG_DIR", str(carpetas_configuracion))

```

1. `Path.cwd()` obtiene la carpeta desde la que se ejecuta el notebook.
2. El operador `/` de `Path` construye la ruta `.yolo_config`.
3. `mkdir(exist_ok=True)` crea la carpeta si no existe y no falla si ya estaba creada.
4. `setdefault` define `YOLO_CONFIG_DIR` solamente si no existía una configuración previa.

Esta preparación se realiza **antes** de importar `ultralytics`, de modo que la biblioteca conozca la ubicación correcta desde el inicio.

5.2. Carga de pesos

```
1 modelo = YOLO("yolov8n.pt")
```

La variable `modelo` queda disponible globalmente para las funciones. Como el archivo `yolov8n.pt` ya está dentro del proyecto, la carga puede realizarse localmente. Si no existiera, Ultralytics intentaría obtenerlo automáticamente.

Los pesos fueron preentrenados con las categorías de COCO. Por eso `modelo.names` relaciona cada identificador numérico con un nombre de clase y `len(modelo.names)` informa cuántas clases puede reconocer el modelo.

5.3. Selección del dispositivo

```
1 dispositivo = 0 if torch.cuda.is_available() else "cpu"
```

`torch.cuda.is_available()` pregunta si PyTorch puede usar CUDA:

- si responde verdadero, se usa 0, es decir, la primera GPU;
- si responde falso, se usa el procesador mediante `cpu`.

Con las versiones CPU instaladas explícitamente en la primera celda, lo normal es que el resultado sea `cpu`. La selección automática permite que el resto del código permanezca igual en ambos casos.

6. Función `abrir_camara`

```
1 def abrir_camara(indice=0):
2     if os.name == "nt":
3         camara = cv2.VideoCapture(indice, cv2.CAP_DSHOW)
4         if not camara.isOpened():
5             camara.release()
6             camara = cv2.VideoCapture(indice)
7     else:
8         camara = cv2.VideoCapture(indice)
9
10    camara.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
11    camara.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
12    return camara
```

El parámetro `indice` identifica la cámara. El valor 0 suele representar la cámara principal; una segunda cámara normalmente se prueba con 1.

En Windows se intenta primero el backend `CAP_DSHOW` (DirectShow). Si no abre, el objeto se libera y se repite el intento con el backend predeterminado. En otros sistemas se utiliza directamente el método normal de OpenCV.

Las llamadas a `camara.set` solicitan una resolución de 1280×720 . Son una petición, no una garantía: el dispositivo puede utilizar la resolución compatible más cercana. Finalmente, la función devuelve el objeto `VideoCapture`; la comprobación definitiva se hace en la función de detección.

7. Función `construir_tabla`

```
1 def construir_tabla(resultado):
2     filas = []
3     cajas = resultado.bboxes
4
5     if cajas is not None:
```

```

6         for numero, (clase, confianza, coordenadas) in enumerate(
7             zip(cajas.cls.tolist(),
8                 cajas.conf.tolist(),
9                 cajas.xyxy.tolist()),
10            start=1,
11        ):
12            x1, y1, x2, y2 = [round(valor)
13                             for valor in coordenadas]
14            filas.append({
15                "N.º": numero,
16                "Objeto": modelo.names[int(clase)],
17                "Confianza": f"{confianza * 100:.1f} %",
18                "Recuadro (x1, y1, x2, y2)":
19                    f"({x1}, {y1}, {x2}, {y2})",
20            })
21
22    return pd.DataFrame(filas)

```

El objeto **resultado** contiene las predicciones de un fotograma. **resultado.boxes** reúne todas las cajas y expone:

Atributo	Contenido
cls	Identificador numérico de la clase de cada detección.
conf	Confianza entre 0 y 1 asociada con cada caja.
xyxy	Coordenadas (x_1, y_1, x_2, y_2) en píxeles.

tolist() mueve los datos a listas normales de Python, lo que facilita su recorrido. **zip** alinea clase, confianza y coordenadas pertenecientes a una misma detección. **enumerate(..., start=1)** agrega una numeración amigable para la tabla.

La confianza se multiplica por 100 y se presenta con un decimal. La clase numérica se convierte en nombre mediante **modelo.names**. Si no hay objetos, **filas** permanece vacía y Pandas devuelve un **DataFrame** sin registros.

7.1. Sistema de coordenadas

(0, 0) se encuentra en la esquina superior izquierda.
 x aumenta hacia la derecha y y aumenta hacia abajo.
 La caja va desde (x_1, y_1) , esquina superior izquierda, hasta (x_2, y_2) , esquina inferior derecha.

8. Función principal: detectar_en_tiempo_real

Esta función coordina la captura, la inferencia, la interfaz y la evidencia. Recibe tres parámetros:

Parámetro	Predeterminado	Uso
nombre_prueba	obligatorio	Texto que identifica la evidencia.
confianza_minima	0.30	Descarta predicciones por debajo del umbral.
indice_camara	0	Selecciona el dispositivo de video.

8.1. Apertura y validación

```

1 camara = abrir_camara(indice_camara)
2 if not camara.isOpened():
3     raise RuntimeError(
4         "No fue posible abrir la cámara..."
5     )

```

La función reutiliza `abrir_camara`. Si la cámara no quedó disponible, lanza una excepción con recomendaciones concretas. Así evita entrar en un bucle que nunca podría recibir imágenes.

8.2. Lectura continua

```

1 while True:
2     lectura_correcta, fotograma = camara.read()
3     if not lectura_correcta:
4         raise RuntimeError(
5             "La cámara se abrió, pero no pudo entregar imágenes."
6         )

```

`camara.read()` devuelve:

- un booleano que indica si la lectura fue correcta;
- el fotograma, representado como un arreglo NumPy.

OpenCV almacena normalmente los canales en orden BGR (azul, verde y rojo), no en RGB. Ese detalle será importante cuando la imagen se envíe a Jupyter.

8.3. Inferencia con YOLO

```

1 resultado = modelo.predict(
2     source=fotograma,
3     conf=confianza_minima,
4     imgsz=640,
5     max_det=100,
6     device=dispositivo,
7     verbose=False,
8 ) [0]

```

Argumento	Significado
<code>source</code>	Imagen que se analizará.
<code>conf</code>	Umbral mínimo de confianza.
<code>imgsz=640</code>	Tamaño de entrada utilizado para la inferencia; YOLO adapta la imagen conservando la geometría necesaria.
<code>max_det=100</code>	Máximo de detecciones permitidas en un fotograma.
<code>device</code>	CPU o GPU seleccionada anteriormente.
<code>verbose=False</code>	Evita imprimir un registro por cada fotograma.

`predict` devuelve una colección porque puede aceptar varias imágenes. El índice `[0]` extrae el resultado correspondiente al único fotograma enviado.

8.4. Dibujo e interfaz

```

1 fotograma_annotado = resultado.plot(
2     line_width=2,
3     font_size=13,

```

```

4 )
5
6 cv2.rectangle(
7     fotograma_annotado,
8     (0, 0),
9     (fotograma_annotado.shape[1], 42),
10    (25, 25, 25),
11    -1,
12 )
13 cv2.putText(
14     fotograma_annotado,
15     "C = capturar evidencia    Q o Esc = salir",
16     (12, 29),
17     cv2.FONT_HERSHEY_SIMPLEX,
18     0.72,
19     (255, 255, 255),
20     2,
21     cv2.LINE_AA,
22 )

```

`resultado.plot()` crea una copia visual con cajas, etiquetas y confianzas. Después, `cv2.rectangle` dibuja una franja oscura en la parte superior. El grosor `-1` indica que el rectángulo se rellena. `cv2.putText` coloca instrucciones blancas y `cv2.LINE_AA` suaviza los bordes de las letras.

El ancho de la franja se obtiene con `fotograma_annotado.shape[1]`, por lo que se adapta al tamaño real del video.

8.5. Lectura del teclado

```

1 cv2.imshow(ventana, fotograma_annotado)
2 tecla = cv2.waitKey(1) & 0xFF
3
4 if tecla in (ord("c"), ord("C")):
5     evidencia = {
6         "nombre": nombre_prueba,
7         "imagen_bgr": fotograma_annotado.copy(),
8         "resultado": resultado,
9     }
10    break
11
12 if tecla in (ord("q"), ord("Q"), 27):
13    break

```

`imshow` actualiza la ventana. `waitKey(1)` concede a OpenCV un instante para procesar eventos y obtiene la tecla. La operación `& 0xFF` normaliza el código a sus ocho bits inferiores.

`ord` convierte una letra en su código numérico y el número 27 representa **Esc**. Al capturar se usa `fotograma_annotado.copy()`; la copia impide que un cambio posterior del arreglo altere la evidencia guardada.

8.6. Liberación segura de recursos

```

1 finally:
2     camara.release()
3     cv2.destroyAllWindows()
4     cv2.waitKey(1)

```


El bloque `finally` se ejecuta tanto al terminar normalmente como al producirse un error. `release` devuelve la cámara al sistema y `destroyAllWindows` cierra las ventanas de OpenCV. La última llamada a `waitKey` ayuda a procesar el evento de cierre en algunos entornos.

Este diseño evita que la cámara quede ocupada y bloquee una prueba posterior.

8.7. Presentación en Jupyter

```

1 imagen_rgb = cv2.cvtColor(
2     evidencia["imagen_bgr"],
3     cv2.COLOR_BGR2RGB,
4 )
5 tabla = construir_tabla(evidencia["resultado"])
6
7 display(Markdown(f"### Evidencia: {nombre_prueba}"))
8 display(Image.fromarray(imagen_rgb))

```

OpenCV usa BGR, mientras que Pillow y Jupyter esperan RGB. `cvtColor` intercambia correctamente los canales para evitar que rojo y azul aparezcan invertidos.

Luego se crea la tabla. Si está vacía, se proponen ajustes de iluminación o confianza. Si tiene registros, se muestra sin el índice automático de Pandas y se informa la cantidad de objetos.

La función finalmente agrega la tabla al diccionario y lo devuelve:

```

1 evidencia["tabla"] = tabla
2 return evidencia

```

La estructura resultante contiene:

- `nombre`: descripción de la prueba;
- `imagen_bgr`: fotograma anotado;
- `resultado`: objeto original producido por Ultralytics;
- `tabla`: resumen tabular de las cajas.

9. Las tres pruebas

Las tres llamadas reutilizan la misma función y solo cambian el nombre de la prueba y, en un caso, el umbral.

```

1 evidencia_prueba_1 = detectar_en_tiempo_real(
2     "Prueba 1 - Persona frente a la cámara",
3     confianza_minima=0.30,
4 )
5
6 evidencia_prueba_2 = detectar_en_tiempo_real(
7     "Prueba 2 - Objeto individual",
8     confianza_minima=0.30,
9 )
10
11 evidencia_prueba_3 = detectar_en_tiempo_real(
12     "Prueba 3 - Dos o más objetos simultáneos",
13     confianza_minima=0.25,
14 )

```

Prueba	Umbral	Objetivo
1	30 %	Capturar una persona frente a la cámara. COCO reconoce person , no el rostro como clase separada.
2	30 %	Detectar un objeto individual perteneciente a las clases conocidas por el modelo.
3	25 %	Capturar al menos dos objetos. El umbral menor aumenta la posibilidad de detectar objetos más difíciles.

Un umbral bajo aumenta la sensibilidad, pero también puede introducir falsos positivos. Un umbral alto exige mayor seguridad al modelo, aunque puede omitir objetos reales.

10. Verificación final

```

1 nombres_variables = [
2     "evidencia_prueba_1",
3     "evidencia_prueba_2",
4     "evidencia_prueba_3",
5 ]
6
7 evidencias = [globals().get(nombre)
8               for nombre in nombres_variables]
```

`globals()` devuelve el espacio de nombres global del notebook. `get(nombre)` recupera cada variable sin provocar un `NameError`; si una prueba no se ejecutó, devuelve `None`.

```

1 if all(evidencia is not None for evidencia in evidencias):
2     cantidades = [len(evidencia["tabla"])
3                   for evidencia in evidencias]
4
5     if (cantidades[0] >= 1
6         and cantidades[1] >= 1
7         and cantidades[2] >= 2):
8         print("Se cumplen las detecciones mínimas.")
```

`all` confirma que las tres capturas existen. Después, `len(evidencia["tabla"])` cuenta las filas detectadas. Se requieren una, una y dos detecciones respectivamente.

Atención. La verificación cuenta objetos, pero no valida sus clases. Por ejemplo, la primera prueba podría aprobar con una botella aunque no aparezca **person**. Una validación más estricta debería comprobar también el contenido de la columna **Objeto**.

11. Parámetros que modifican el comportamiento

Parámetro	Si disminuye	Si aumenta
<code>confianza_minima</code>	Detecta más candidatos, incluidos posibles errores.	Muestra menos resultados, pero generalmente más seguros.
<code>imgsz</code>	Puede aumentar la velocidad y perder objetos pequeños.	Puede mejorar detalle, con mayor costo de tiempo y memoria.
<code>max_det</code>	Limita escenas muy cargadas.	Permite más cajas y requiere más procesamiento posterior.

Parámetro	Si disminuye	Si aumenta
<code>indice_camara</code>	No representa calidad; selecciona otro dispositivo.	Prueba cámaras enumeradas con otros índices.
<code>CAP_PROP_FRAME_WIDTH</code> y <code>HEIGHT</code>	Reduce los datos capturados.	Solicita más resolución, si la cámara la admite.

12. Manejo de errores y robustez

El código considera varias situaciones:

- intenta un backend alternativo si `DirectShow` no abre;
- verifica por separado que la cámara se haya abierto y que entregue fotogramas;
- usa `try/finally` para liberar el dispositivo;
- acepta mayúsculas y minúsculas para las teclas;
- devuelve `None` cuando el usuario sale sin capturar;
- maneja una tabla vacía sin asumir que hubo detecciones;
- recupera evidencias con `globals().get` para que la verificación no falle si falta una variable.

Los mensajes de solución de problemas del notebook orientan sobre permisos, aplicaciones que ocupan la cámara, iluminación, índice de cámara, velocidad y reinicio del kernel.

13. Limitaciones y mejoras recomendadas

13.1. Limitaciones actuales

1. **Las evidencias no se exportan como archivos.** Permanecen en memoria y en las salidas del notebook; es necesario guardar el notebook para conservarlas.
2. **La validación final solo cuenta cajas.** No comprueba que la primera escena contenga realmente una persona ni que la segunda corresponda al objeto solicitado.
3. **Las clases aparecen en inglés.** Son los nombres originales de COCO proporcionados por el modelo.
4. **Solo se usa la primera GPU.** El valor 0 no contempla selección entre varias GPU.
5. **NumPy se importa pero no se referencia directamente.** Las imágenes sí son arreglos NumPy, pero el alias `np` podría eliminarse sin cambiar el comportamiento actual.
6. **La interfaz requiere escritorio local.** `cv2.imshow` no es adecuada para un servidor sin pantalla o un notebook remoto.

13.2. Mejoras posibles

- Guardar automáticamente cada imagen con `cv2.imwrite` y exportar la tabla a CSV.
- Validar clases específicas, por ejemplo exigir `"person"` en la primera tabla.
- Traducir los nombres mediante un diccionario español.
- Calcular fotogramas por segundo para medir rendimiento.
- Permitir seleccionar resolución, dispositivo y umbral desde controles del notebook.

- Agregar fecha y hora a cada evidencia.
- Separar la lógica de inferencia de la interfaz para facilitar pruebas automáticas.

14. Cómo ejecutar correctamente el notebook

1. Abrir `NombreApellido_DeteccionObjetos.ipynb` en Jupyter, JupyterLab o VS Code.
2. Ejecutar la instalación una vez y reiniciar el kernel.
3. Ejecutar la celda de importaciones y confirmar que el modelo se cargó.
4. Ejecutar la celda que define las tres funciones.
5. Ejecutar cada prueba por separado, dar foco a la ventana y presionar **C** cuando la detección sea visible.
6. Comprobar que la imagen y la tabla aparecen debajo de cada celda.
7. Ejecutar la verificación final.
8. Guardar el notebook con todas sus salidas.
9. Antes de entregar, sustituir `NombreApellido` en el archivo y en el encabezado por los datos reales del estudiante.

15. Resumen conceptual

El programa sigue una separación de responsabilidades clara:

- `abrir_camara` se ocupa del dispositivo físico;
- `construir_tabla` transforma resultados del modelo en datos comprensibles;
- `detectar_en_tiempo_real` controla el ciclo completo y la interacción;
- las tres celdas de prueba reutilizan esa lógica;
- la última celda comprueba que las evidencias mínimas existen.

La secuencia esencial puede resumirse así:

capturar imagen → inferir con YOLO → dibujar detecciones → mostrar → capturar evidencia

El resultado es una aplicación educativa, local y reutilizable que conecta una cámara con un modelo de aprendizaje profundo y convierte sus predicciones en una salida visual y tabular.

Glosario breve

Clase: categoría predicha para un objeto.

Confianza: puntuación con la que el modelo respalda una detección.

Fotograma: una imagen individual de la secuencia de video.

Inferencia: uso de un modelo entrenado para producir predicciones.

Peso: parámetro aprendido por la red neuronal y almacenado en el archivo `.pt`.

Recuadro delimitador: rectángulo que localiza un objeto.

Umbral: valor mínimo aceptado para conservar una predicción.